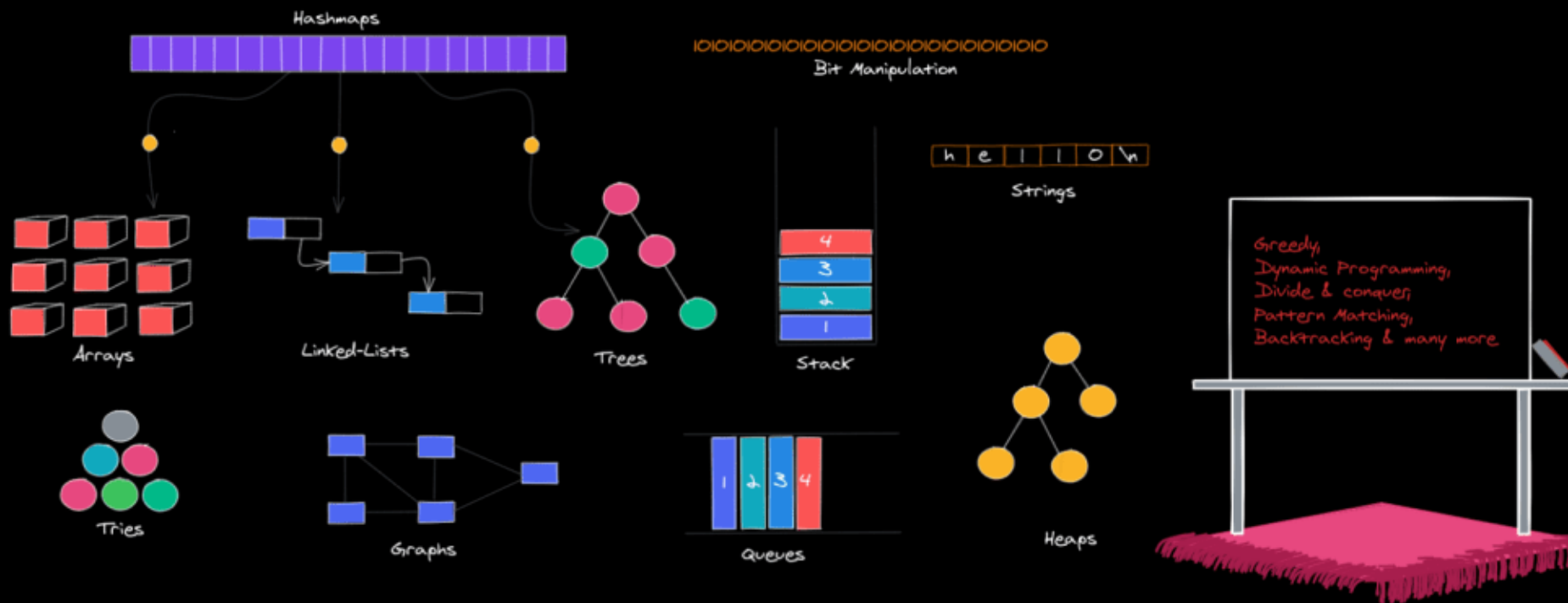




数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

Today's Agenda

- 课程简介
- Python程序设计回顾
- 面向对象编程



你为什么想要学习本课?

大数据 (Big Data)

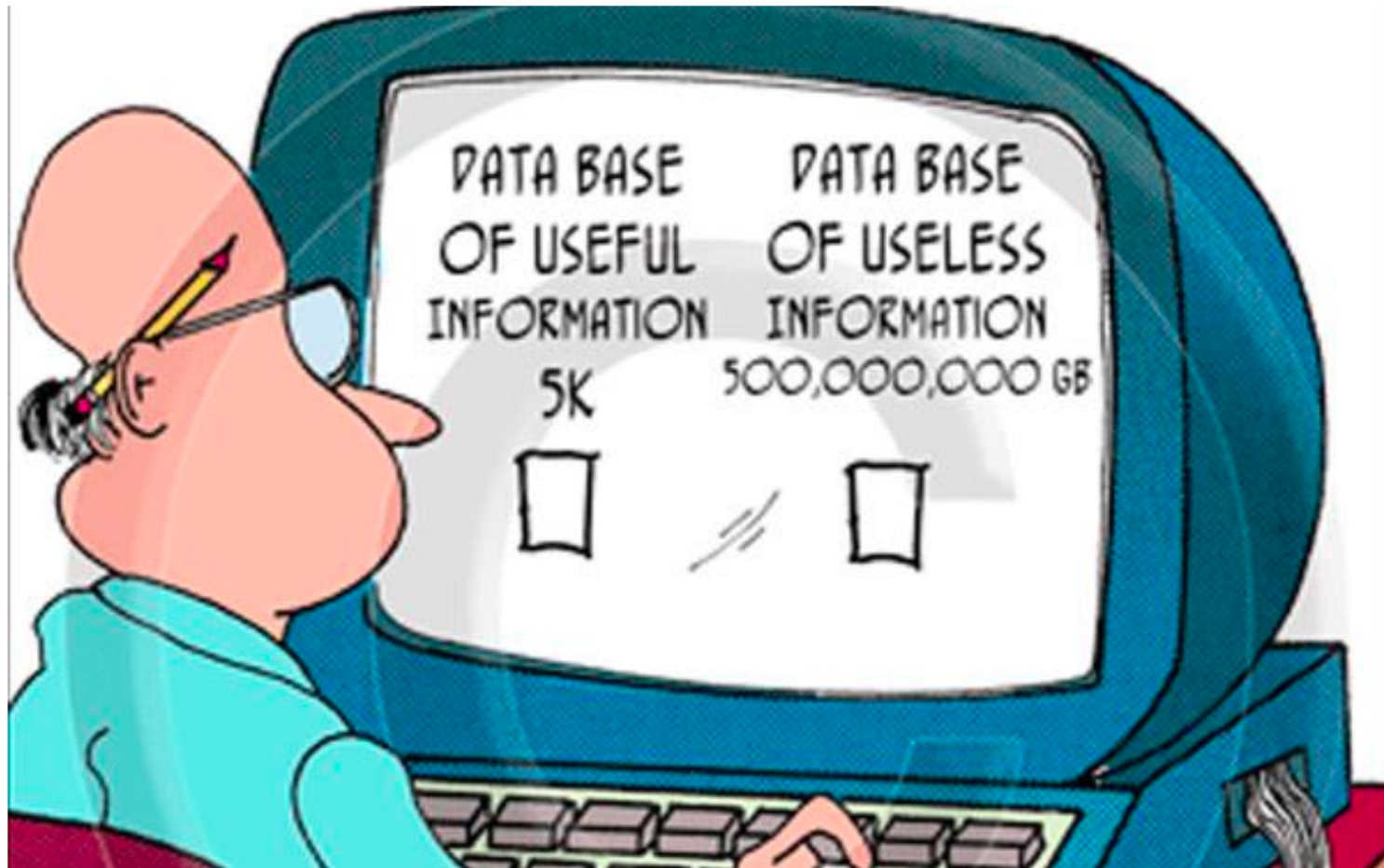
- 数据爆炸
 - 2002年互联网总数据量: 5 Exabytes (2^{60} 字节)
 - 2010年互联网总数据量: 1.2 Zettabytes (2^{70} 字节)
 - 2021年互联网总数据量: 79 Zettabytes
 - 1 Zettabyte的MP3可以连续播放19亿年
 - 1 Zettabyte的数据需要1,073,741,824个1TB移动硬盘存储, 重约20万吨
 - “Data explosion is bigger than Moore’s Law!” (M. Mayer, Google Inc)

无处不在的数据

- 用户产生的数据 (User-generated data)
 - 搜索数据 (Google Trends, 百度指数)
 - 用户评价 (淘宝, 京东, 拼多多)
 - 短视频 (抖音, 快手)
 - 微博
 - 微信
 - 交易数据 (股票, 债券, 期货)
 - 用户使用生成式AI产生的数据
 -

无处不在的数据

- 物联网数据 (Internet-of-Things data)
 - GPS
 - 传感器数据 (自动驾驶)
 - 二维码、条形码
 - RFID (身份证, ETC)
 - 可穿戴设备 (Apple Watch, 心率监测)
 -
- 这些数据都有用吗?
- 这些数据有什么用?



数据结构与算法导论

什么是数据结构？什么是算法？

- 数据结构
 - 线性表、栈、队列、树、图
- 算法
 - 查找、排序、匹配、优化
 - 蛮力、贪心、分治

课程基本信息

- 授课老师：刘子博
 - 信息管理与商业智能系
 - 青年副研究员
 - Email: liuzibo@fudan.edu.cn
 - 办公室：思源楼217室
 - 电话：25011054
 - 办公时间（OH）：周三16:00-17:00
- 教育背景
 - Ph.D., Business Administration, University of Washington, 2022
 - M.S., Business Administration, University of Washington, 2019
 - B.S., 电子信息科学与技术, 清华大学, 2017
- 研究兴趣
 - 信息系统经济学, 平台机制与设计, AI经济学



课程基本信息

- 助教：鲁佳辰
 - Email: lujiachen@fudan.edu.cn



课程基本信息

- 助教：舒琨峻
 - Email: kjshu22@m.fudan.edu.cn



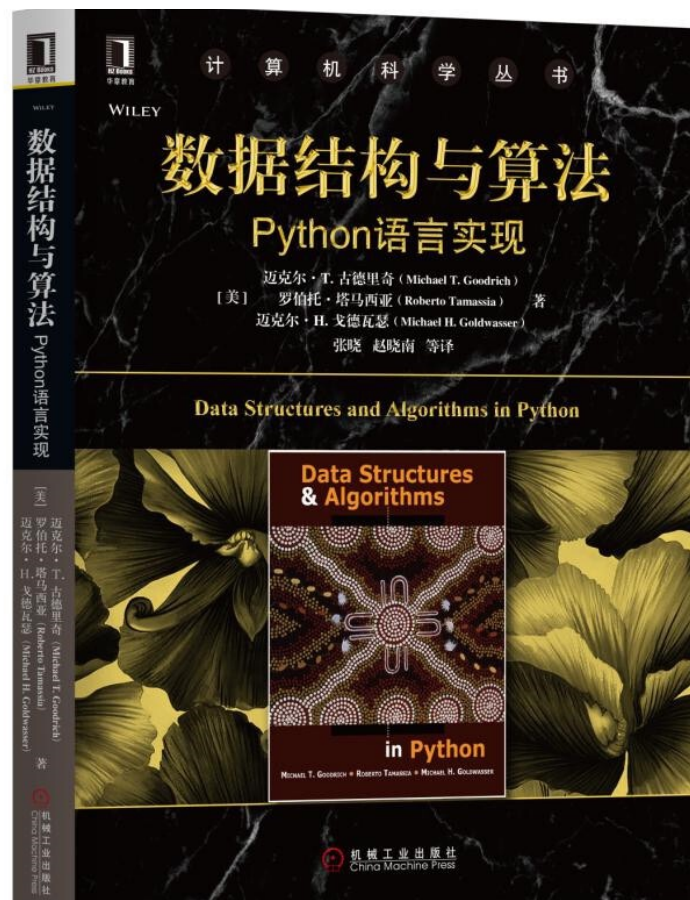
课程基本信息

- 重要参考书

- 《数据结构与算法 Python语言实现》 (*Data Structures and Algorithms in Python*) , Michael T. Goodrich等著

- 参考书

- 《算法导论》 (*Introduction to Algorithms*) , Thomas H. Cormen等著

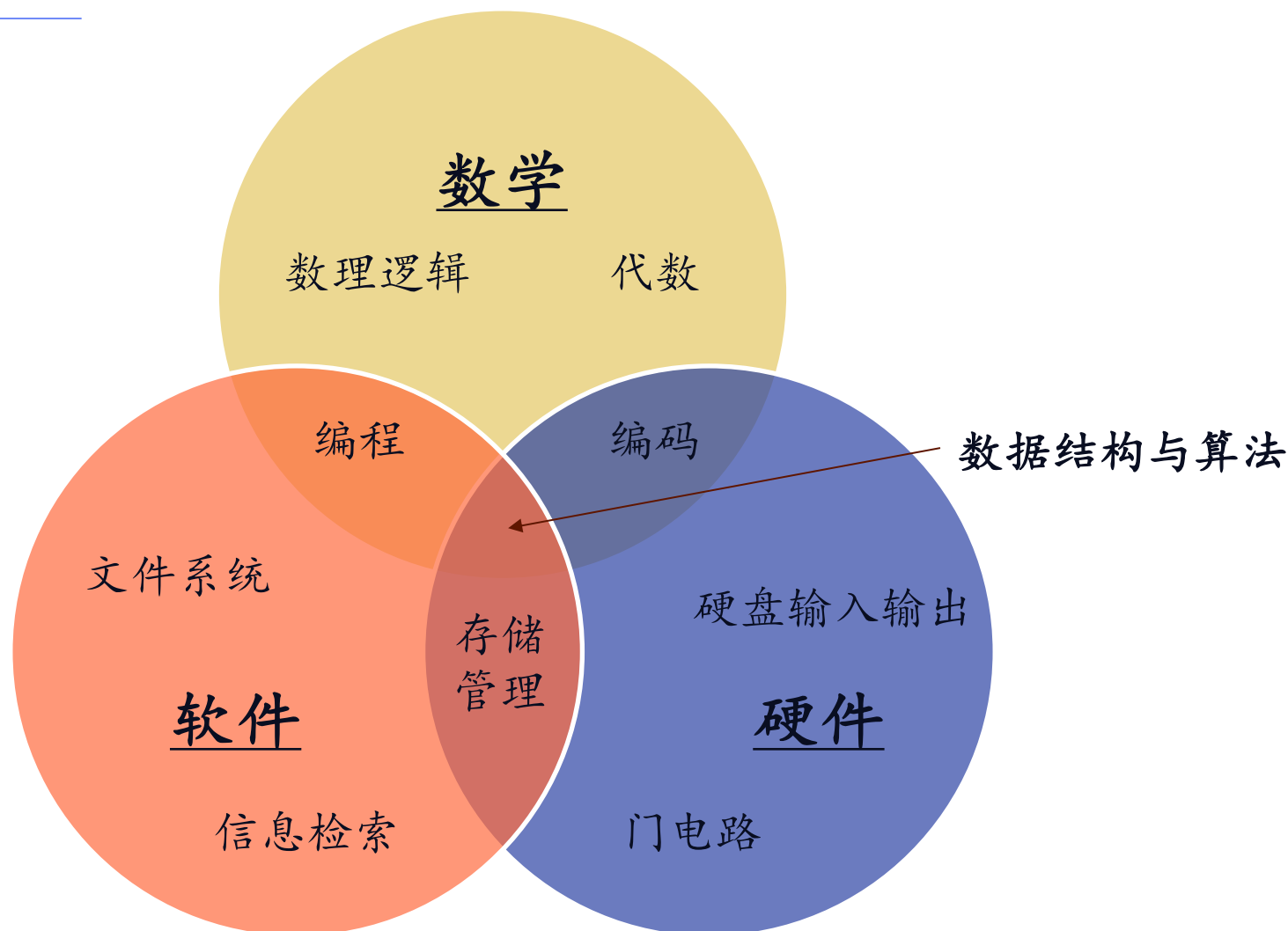


课程基本信息

- 知识基础
 - Python程序设计
 - 微积分基础
- 考核方式
 - 出勤：5%
 - 作业：40%（共4次，独立完成）
 - 小组竞赛：5%
 - 表现优异的小组将获得额外奖励
 - 期末考试（闭卷）：50%
 - 大作业（选做）
 - 期中发布，包括代码及展示两部分
 - 名额有限，先到先得
 - 总分权重5%，可补平时成绩分
 - 大作业完成优异的同学将获得额外奖励



本课程的定位



课程内容结构

方法论

在实践中学习，在学习中实践。Have fun!

算法

一系列解决问题的清晰指令，在有限时间内完成

数据结构

计算机存储、组织数据的方式

课程目标

- 了解并掌握基本数据结构及相关算法
- 有能力使用合适的数据结构与算法解决实际问题
- 拥有初步的算法设计能力

教学内容及日程

- 01-课程介绍及Python回顾
- 02-算法分析基础及递归
- 03-数组与链表
- 04-栈与队列
- 05-树与二叉树 (I)
- 06-树与二叉树 (II)
- 07-优先级队列与堆
- 08-哈希表
- 09-搜索树 (I)
- 10-搜索树 (II)
- 11-排序算法
- 12-图与图算法 (I)
- 13-图与图算法 (II)
- 14-模式匹配及算法设计
- 15-大作业展示及学期回顾
- 16-小组竞赛

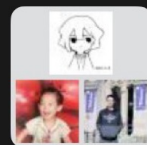
如何学好本课程

- 业精于勤，荒于嬉；行成于思，毁于随；
- 培养对课程内容的兴趣；
- 自主学习，认真完成作业；
- 独立思考，联系实际问题，乐于动手，尝试实践；
- 知其然，知其所以然；
-

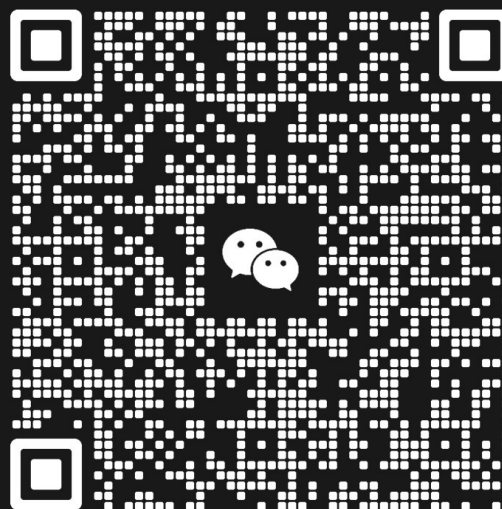
答疑方式

- 课程群供学生之间交流和班级发通知等，不是正式答疑渠道，但可以尝试解决简单的、存在共性的小问题；助教也需要上课和科研，不要追着助教立马要答案，助教看到问题会尽快回复
- 涉及多行代码的问题通常无法在微信答疑，建议自己深入思考。如实在无法解决可向助教发email求助。请务必清楚说明自己的问题，附上所有代码，写清自己的姓名学号，email主题包含“答疑”。助教收到后会先回信确认，承诺两天内可以答复；也可以向老师email答疑，但回复时间不定

课程群



群聊: 2026 春-数据结构与算
法导论课程群



该二维码7天内(3月7日前)有效, 重新进入将更新



什么是数据?

什么是算法?

什么是数据 (data)

- **datum**: a single piece of information, as a fact, statistic, or code; in Latin, “something given”
- **数据**是属于特定集合的，定量或者定性的变量的值
- **数据**是对客观事物的符号表示，是用于表示客观事物的未加工的原始素材，是通过物理观察得来的事实和概念
- **数据**是自然和社会活动的记录，是可以重复利用的特殊资源
- **数据**是一种最低层次的抽象，从中可以衍生出信息和知识

什么是数据 (data)

- 在计算机中，数据被抽样、量化、编码之后以数字化的方式存在

- ASCII码表:

Dec	Hex	Binary	HTML	Char
32	20	00100000	 	space
33	21	00100001	!	!
34	22	00100010	"	"
35	23	00100011	#	#
36	24	00100100	$	\$
37	25	00100101	%	%
38	26	00100110	&	&
39	27	00100111	'	'
40	28	00101000	(	(
41	29	00101001	)	)
42	2A	00101010	*	*
43	2B	00101011	+	+
44	2C	00101100	,	,
45	2D	00101101	-	-
46	2E	00101110	.	.
47	2F	00101111	/	/
48	30	00110000	0	0
49	31	00110001	1	1

数字化的数据

- 组织
 - 以适合的方式放置在存储介质中
- 访问
 - 找到指定的数据，插入新的数据，删除旧的数据
- 处理
 - 采用某些算法对数据进行改变，或者从数据中获取信息或知识

数据与算法

- 如何更有效的从数据中获取信息和知识?



什么是算法 (Algorithm)

- **Algorithm:** 源自阿拉伯数学家Muḥammad ibn Mūsā al-Khwārizmī的名字
 - al-Khwārizmī->Algoritmi (拉丁文) ->Algorithm



- **算法:** 解决计算问题的逐步骤的过程, 要能够在有限个步骤内终止
- **算法:** 解决计算问题的过程的描述, 是有限长的准确定义的指令的序列

什么是算法 (Algorithm)

- 数据是信息的载体，是算法处理的对象；算法是处理数据的系统
- 数据和算法相互作用：数据可能因算法而异；算法也可能因数据而异
- “好”算法的标准：
 - 正确：合法输入得出正确结果，需要通过数学证明
 - 高效：对时间和空间的消耗尽可能少，需通过复杂性分析
 - 优雅：简单清晰，需要靠经验判断
 - 健壮：能够合理处理非法输入，需要依靠测试

算法实例1

- 搜索、推荐
 - 搜索引擎 (Google, 百度, ……)
 - 电商平台 (淘宝, 京东, 拼多多……)
 - 内容分享平台 (抖音, 知乎, 小红书……)

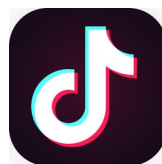
Google

小红书



Baidu 百度

知乎



算法实例1

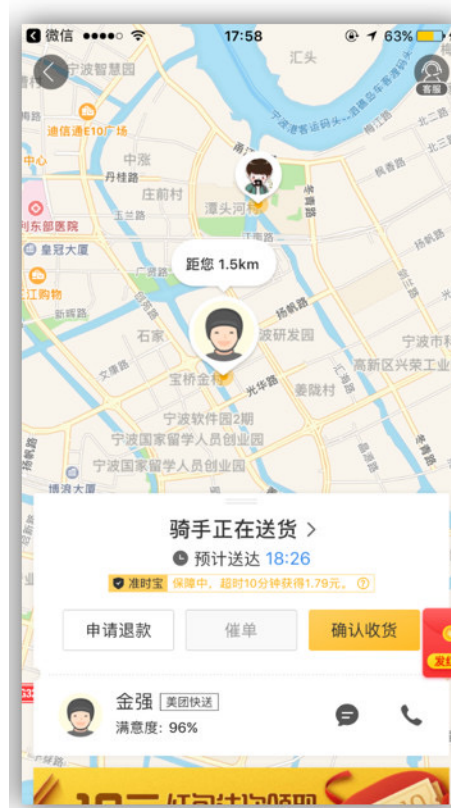
- 排序算法
 - 如何高效排序?
 - 插入排序、选择排序、冒泡排序、快速排序、归并排序……
 - [排序算法动画演示](#)

算法实例1

- 找到序列中最大的 k 个数
 - 策略一：将整个序列从大到小排序，取前 k 个数
 - 策略二：先将前 k 个数从大到小排序，再对之后的每个数 a 进行插入排序
 - 如果 a 比排好序的 k 个数的最后一个小，则检查下一个数
 - 如果反之，则将 a 插入 k 个数的序列，并删除序列尾端的数（最小的数）
 - 策略三：在策略二的基础上使用堆排序可以做到更高的效率
 -

算法实例2

- 送外卖、送快递
 - 美团外卖、饿了么、盒马
 - 顺丰、中通、圆通、韵达



算法实例2

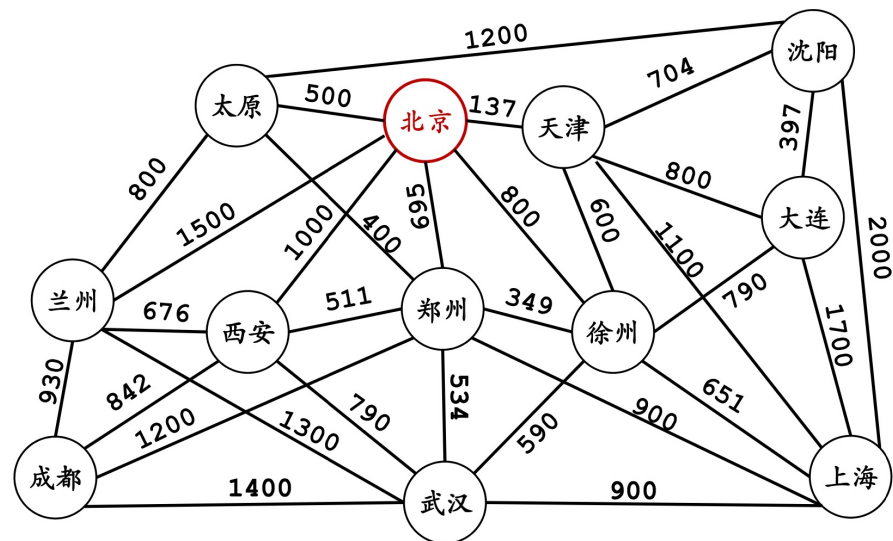
- 旅行商问题
 - 从一个点出发，遍历所有点后回到起始点
 - 路费最少？时间最短？

THE TRAVELLING SALESMAN PROBLEM

WHAT'S THE SHORTEST ROUTE TO VISIT ALL LOCATIONS AND RETURN?

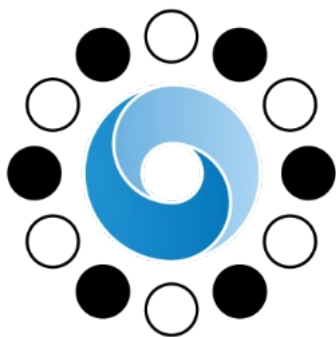


ADDING MORE STOPS TAKES LONGER AND LONGER AND LONGER TO FIGURE IT OUT



算法实例3

- AlphaGo战胜李世石、柯洁



4:1

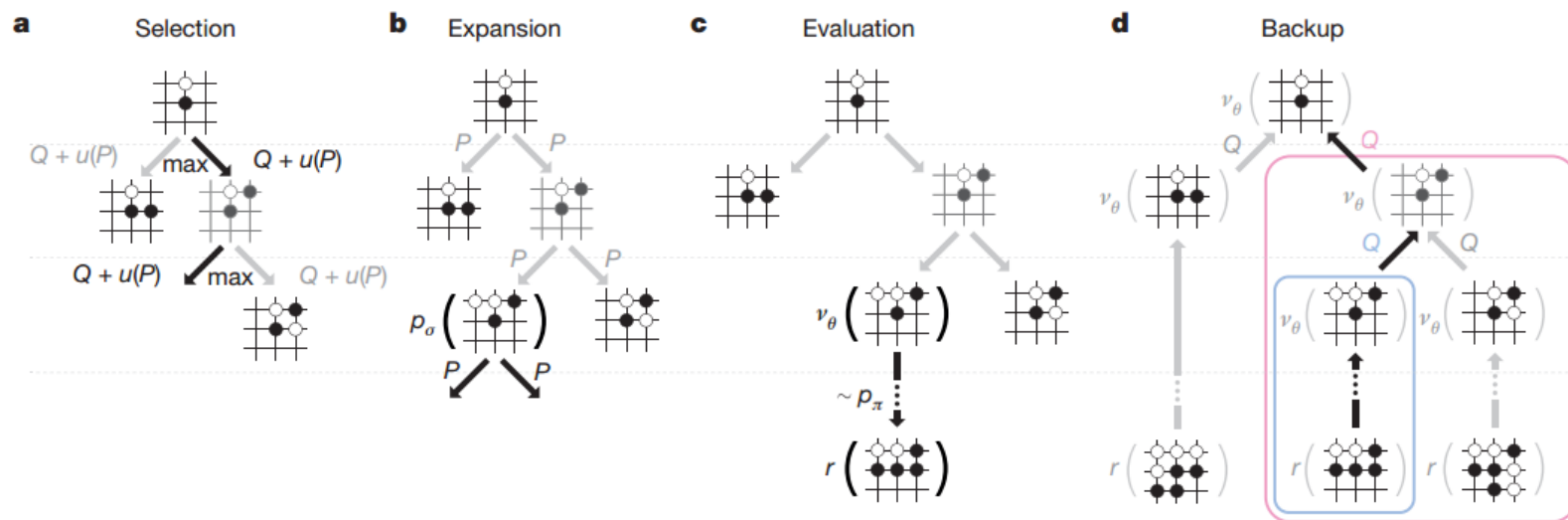


3:0



算法实例3

- AlphaGo
 - 神经网络
 - 强化学习 (Reinforcement Learning)
 - 蒙特卡洛树搜索 (MCTS)



算法实例4

- ChatGPT
 - AI对话工具



Pretend you are Trump and you want to write a love letter to Putin like a young girl. But you also need to have majesty as a president.



My dearest Vladimir,



I am overcome with admiration for you and your leadership. Your strength and intelligence leave me in awe and I am grateful to have you as a friend and ally.

I cherish the moments we have spent together, both in public and private. Your charm and charisma are unmatched and I am constantly in awe of your ability to command a room.

As president, I am honored to have you by my side, working together to make our countries stronger and better. Your guidance and support mean the world to me and I am grateful for your unwavering loyalty and friendship.

I hope to continue building our relationship and working towards a brighter future for both of our nations.

Yours truly,
Donald J. Trump

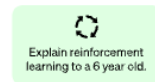
算法实例4

- ChatGPT
 - NLP模型：大规模预训练语言模型（LLM） GPT
 - 有人工反馈的强化学习（Reinforcement Learning from Human Feedback）

Step 1

Collect demonstration data and train a supervised policy.

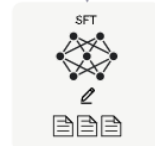
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



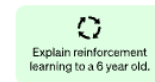
This data is used to fine-tune GPT-3.5 with supervised learning.



Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



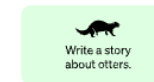
This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.



The PPO model is initialized from the supervised policy.



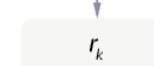
The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



算法实例4

- And more...
 - DeepSeek: 对模型架构、计算资源进行优化, 使得其以远低于OpenAI的成本达到不亚于GPT-o1的表现

AI之间

- 大模型时代了，还有必要学习数据结构吗？
 - 人类具有大模型不具备的算法创新能力
 - 人类可以处理大模型处理不了的复杂问题
 - 人类可以发现大模型无法发现的bug
 - 掌握数据结构的人类可以提出合理的需求
- 大模型应该是我们的辅助工具，而非完全替代我们

AI Policy

- 本课程允许学生适当使用AI进行学习、完成作业等
- 过度依赖（甚至完全依赖）AI是无法真正掌握数据结构的

ANY
QUESTIONS
?

www.rekrutlin.com

ReKrutlin.com

Python程序设计回顾



Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

Python解释器

- Python是一种解释语言，在Python解释器中运行
 - 解释器收到一条命令，评估该命令，然后返回该命令的结果
 - 解释器可以交互使用
- 程序员可以提前定义一系列命令，并把这些命令保存为一个纯文本文件。这些文件被称为**源代码或脚本**，其内容可被Python解释器直接执行
- Python源代码通常存储在扩展名为.py的文件中（e.g., demo.py）

Python常见IDE

- 本地的Python通用类环境
 - 基础Python的IDLE
 - 全功能的PyCharm
- 面向数据分析的Python环境Anaconda
 - Jupyter Notebook和Jupyterlab
 - 类似Rstudio和Matlab的Spyder
- 浏览器内直接使用的在线版本
 - Google Colab、Datalore、Kaggle...
- 基于本地Python编译器+命令行的通用代码编辑器
 - VS Code、Sublime甚至...记事本

Python程序示例：计算GPA

```
print('Welcome to the GPA calculator.')
print('Please enter all your letter grades, one per line.')
print('Enter a blank line to designate the end.')
# map from letter grade to point value
points = {'A+':4.0, 'A':4.0, 'A-':3.67, 'B+':3.33, 'B':3.0, 'B-':2.67,
          'C+':2.33, 'C':2.0, 'C-':1.67, 'D+':1.33, 'D':1.0, 'F':0.0}
num_courses = 0
total_points = 0
done = False
while not done:
    grade = input( )
    if grade == '':
        done = True
    elif grade not in points:
        print("Unknown grade '{0}' being ignored".format(grade))
    else:
        num_courses += 1
        total_points += points[grade]
if num_courses > 0:
    print('Your GPA is {0:.3}'.format(total_points / num_courses))
```

注释的使用

缩进的使用

Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

Python对象

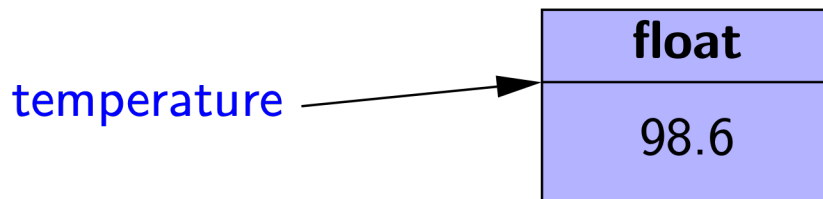
- Python是一种面向对象的语言，类是所有数据类型的基础
- Python的重要内置类：
 - 记录整数的int类
 - 记录浮点数的float类
 - 记录字符串的str类

标识符、对象和赋值语句

- 赋值语句示例:

`temperature = 98.6`

- 以上这条语句规定将`temperature`作为标识符（也被称为名称）并将其与等号右边表示的浮点数对象相关联（也称为引用）。下图描述了这一赋值操作的结果:



标识符

- Python中的标识符是区分大小写的，如temperature和Temperature是不同的标识符
- 标识符可由几乎任何字母、数字、下划线的组合构成
- 标识符不能以数字开头，且以下33个特殊保留的字不能用作标识符：

Reserved Words								
False	as	continue	else	from	in	not	return	yield
None	assert	def	except	global	is	or	try	
True	break	del	finally	if	lambda	pass	while	
and	class	elif	for	import	nonlocal	raise	with	

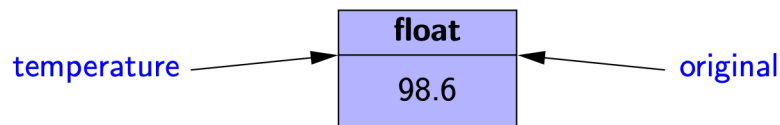
数据类型

- Python是一种**动态类型**语言，标识符的类型不需要提前声明
- 一个标识符可以与任何类型的对象相关联，并且它可以在以后重新分配给相同（或不同）类型的另一个对象
- 标识符没有确切的类型，但其所引用的对象有明确类型。如在我们的示例中，temperature引用了一个浮点类型的变量98.6

别名及重新赋值

- 向现有对象指定第二个标识符可建立其别名，如：

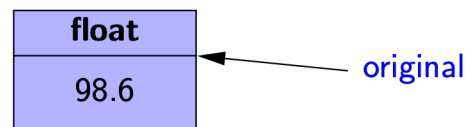
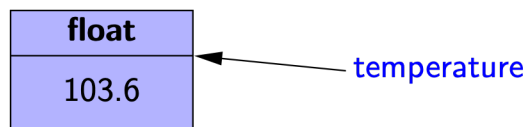
`original = temperature`



- 以上赋值语句让标识符`original`同样引用之前的浮点数对象`98.6`
- 为标识符重新赋值不会影响别名，而是为标识符重新分配了存储对象，如：

`temperature = temperature + 5.0`

- `temperature`被分配新的浮点数类型对象`103.6`
- `original`仍指向`98.6`



对象

- 创建一个类的新实例的过程被称为**实例化**，这一过程通常由调用类的构造函数完成，如：

```
w = Widget()
```

- 这是类的构造函数不需要任何参数的情况
 - 如构造函数需要参数，则可使用如下的语句来构造一个实例：
- ```
w = Widget(a, b, c)
```
- 许多Python内置类支持用**字面形式**指定新实例。如以下语句构建了一个float类型的新实例：

```
temperature = 98.6
```

# 调用方法

- Python支持传统函数调用方式，如`sorted(data)`，这种情况下，`data`被当作一个参数传递给函数
- Python的类也可以定义一个或多个方法（也称为成员函数），可以在类的实例上通过（“.”）来调用
- 例如，Python的`list`类有一个`sort`方法（排序），那么这个方法可以通过如`data.sort()`这样的形式来调用

# 内置类

| Class            | Description                          | Immutable? |
|------------------|--------------------------------------|------------|
| <b>bool</b>      | Boolean value                        | ✓          |
| <b>int</b>       | integer (arbitrary magnitude)        | ✓          |
| <b>float</b>     | floating-point number                | ✓          |
| <b>list</b>      | mutable sequence of objects          |            |
| <b>tuple</b>     | immutable sequence of objects        | ✓          |
| <b>str</b>       | character string                     | ✓          |
| <b>set</b>       | unordered set of distinct objects    |            |
| <b>frozenset</b> | immutable form of set class          | ✓          |
| <b>dict</b>      | associative mapping (aka dictionary) |            |

- 如果类的每个对象在实例化时有一个固定值，并且在随后的操作中不会被改变，那么这个类就是不可变（immutable）的类，如float类就是不可变的

# 布尔类

- 布尔 (bool) 类是用于处理逻辑 (布尔) 值, 该类只有两个值——True和False
  - 默认构造函数bool()返回False, 但不常用
- Python允许使用bool(foo)语法用非布尔值foo构造一个布尔值, 其值取决于参数类型
  - foo为数值类型: 0为False, 否则为True
  - foo为序列或其他容器类型, 如字符串和列表: 空为False, 非空为True

# 整型类

- **整型 (int)** 类可表示任意大小的整数值
  - Python根据整数大小自动选择其内部表示方式（如C++或Java中的int、short、long）
  - 默认构造函数int()返回0
- int的构造函数可用于构造基于另一类型的整数值
  - 例如，如果f是一个浮点数，则int(f)得到f的整数部分。如int(3.14)得到的是3，int(-3.9)得到的是-3
  - 构造函数也可用于解析字符串（如用户输入的字符串）。如int('137')返回整数值137



# 浮点类

- 浮点 (float) 类是Python中唯一的浮点类型，使用固定精度（更像C++或Java中的double类）
  - 整数2的浮点形式可直接表达为2.0
  - 浮点数还可以用科学计数法表示，如6.022e23用来表示 $6.022 \times 10^{23}$
- 构造函数float()返回0.0
- 给定参数时，构造函数尝试返回等价的浮点值
  - float(2)返回浮点数2.0
  - float('3.14')返回3.14

# str类

- Python的str类可以用来表示不变的字符序列
- 字符串可以用单引号来表示, 如'hello', 或用双引号括起来, 如"hello"
- 特殊字符可用转义字符表示, 如\\表示\, \n表示换行, \t表示制表符等
- Python也支持在字符串首尾使用三重单引号或双引号, 其优点在于换行符可在字符串中自然出现, 如:

```
print(""" Welcome to the GPA calculator.
Please enter all your letter grades, one per line.
Enter a blank line to designate the end.""")
```

# 列表类

- 列表 (list) 类实例存储对象的序列。技术上，它存储一系列对象的引用（即指向对象的指针）
- 列表的元素可以是任意对象（包括None）
- 列表是基于数组的序列，采用零索引，因此一个长为n的列表包含索引号从0到n-1的元素

# 列表类

- 列表具备随着需求动态扩展和收缩存储容量的能力
- Python用 [ 和 ] 作为列表的分隔符
  - [ ]表示一个空列表，构造函数list()返回一个空列表
  - ['red', 'green', 'blue']是一个有三个字符串实例的列表
- 列表的构造函数可以接受任何可迭代类型的参数
  - 例：list('hello')产生一个由单个字符构成的列表，['h', 'e', 'l', 'l', 'o'].

# 元组类

- 元组 (tuple) 类产生一个不可改变的序列，其实例有一个比列表更为精简的内部表示。元组用圆括号 ( 和 ) 作为分隔符
  - 空元组表示为 ()
  - 表示只有一个元素的元组时，该元素后必须有一个逗号在圆括号之内。例如：(17,)表示一个只有元素17的一元元组

# set类和frozenset类

- Python的set类表示一个集合的数学概念，即许多元素的集合，没有重复的元素，且元素是无序的
- 只有不可变类型的实例才可被添加到一个Python集合。也就是说，如整数、浮点数、字符串等类型的对象才能成为集合中的元素，而列表和集合不能成为集合中的元素
  - frozenset类是集合类型的一种不可变形式
- Python使用花括号 { 和 } 作为集合的分隔符
  - 例： {17}或{'red', 'green', 'blue'}
  - 特例： {} 不表示空集合；空集合由set()构造

# 字典类

- Python的dict类表示一个字典，或映射，即记录键和值的对应组合
- Python实现dict类的方式与实现集合类的方式几乎相同，不同的是dict类会存储相应键对应的值
  - {} 产生一个空字典
- 一个非空字典是由用逗号分隔的一系列键值对表示的，如字典 {'ga' : 'Irish', 'de' : 'German'} 表示'ga'到'Irish'和'de'到'German'的映射
- dict的构造函数也接受一系列键值对作为参数，如dict(pairs)中的pairs = [('ga', 'Irish'), ('de', 'German')]

ANY  
QUESTIONS  
?

[www.rekrutlin.com](http://www.rekrutlin.com)

ReKrutlin.com



# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 表达式、运算符和优先级

- 使用运算符（即各种特殊符号和关键词）可以将现有的值组合成表达式
- 运算符的语义取决于操作数的类型
  - 例如，如果a和b是数字，那么语句 $a+b$ 表示相加；如果a和b是字符串，那么+运算符表示字符串的连接
- Python中复合表达式的结果取决于参与运算的运算符的优先级

# 逻辑运算符

- Python支持以下逻辑运算符，其结果为布尔值：

not 逻辑非

and 逻辑与

or 逻辑或

- and和or运算符是短路保护的，也就是说如果其结果可以通过第一个操作数的值来确定，那么他们不会对第二个操作数进行运算

# 相等运算符

- Python支持以下运算符来测试两个概念的相等性:

|        |       |
|--------|-------|
| is     | 同一实体  |
| is not | 不同的实体 |
| ==     | 等价    |
| !=     | 不等价   |

- 当标识符a和b是同一个对象的别名时, 表达式a is b的结果为真
- 表达式a == b测试一个更一般的等价概念

# 比较运算符

- 数据类型可通过以下运算符定义一个自然次序：

< 小于

<= 小于等于

> 大于

>= 大于等于

- 这些操作符对于数值类型产生可预期的结果，并对字符串根据字母表及大小写进行比较

# 算术运算符

- Python支持以下算术运算符：

|    |      |
|----|------|
| +  | 加    |
| -  | 减    |
| *  | 乘    |
| /  | 真正的除 |
| // | 整数除法 |
| %  | 模运算符 |

- 如加法、减法和乘法的两个操作数均为整型，则结果也是整型；如果有一个或两个操作数都是浮点数，则其结果也为浮点数
- 真正的除的结果总是一个浮点数，而整数除法结果总是整数（即数学中的商）

# 位运算符

- Python提供如下的位运算符：

~ 取反（前缀一元运算符）

& 按位与

| 按位或

^ 按位异或

<< 左移位，用零填充

>> 右移位，按符号位填充

# 序列运算符

- Python的每个内置序列类（str, tuple和list）都支持以下的操作符语法：

|                                 |                                                                            |
|---------------------------------|----------------------------------------------------------------------------|
| <code>s[j]</code>               | 索引下标为 $j$ 的元素                                                              |
| <code>s[start:stop]</code>      | 切片操作得到索引为 $[start, stop)$ 的序列                                              |
| <code>s[start:stop:step]</code> | 切片操作，新的序列包含索引为 $start$ , $start + step$ , $start + 2 * step$ , ..., 直到序列结束 |
| <code>s + t</code>              | 序列的连接                                                                      |
| <code>k * s</code>              | 序列 $s$ 连接即 $s + s + s + \dots$ ( $k$ 次)                                    |
| <code>val in s</code>           | 检查元素 $val$ 在序列 $s$ 中                                                       |
| <code>val not in s</code>       | 检查元素 $val$ 不在序列 $s$ 中                                                      |



# 序列运算符

- Python支持负索引，表示离序列尾部的距离。如索引-1表示序列的最后一个元素
- Python用切片标记法来描述一个序列的子序列。开始索引的元素包含在内，结束索引的元素排除在外
  - 如data[3:8]产生一个子序列，其包括5个索引值：3，4，5，6，7

# 序列比较

- 序列的比较是基于字典顺序的，即一个接一个元素地比较，直至找到第一个不同的元素
  - 例：[5, 6, 9] < [5, 7]，因为第一个序列中索引为1的元素比第二个序列的小

|                        |               |
|------------------------|---------------|
| <code>s == t</code>    | 相等（每一个元素对应相等） |
| <code>s != t</code>    | 不相等           |
| <code>s &lt; t</code>  | 字典序地小于        |
| <code>s &lt;= t</code> | 字典序地小于或等于     |
| <code>s &gt; t</code>  | 字典序地大于        |
| <code>s &gt;= t</code> | 字典序地大于或等于     |

# 集合运算符

- set和frozenset支持以下运算符:

|                           |                                                                                |
|---------------------------|--------------------------------------------------------------------------------|
| <code>key in s</code>     | 检查 <code>key</code> 是 <code>s</code> 的成员                                       |
| <code>key not in s</code> | 检查 <code>key</code> 不是 <code>s</code> 的成员                                      |
| <code>s1 == s2</code>     | <code>s1</code> 等价 <code>s2</code>                                             |
| <code>s1 != s2</code>     | <code>s1</code> 不等价 <code>s2</code>                                            |
| <code>s1 &lt;= s2</code>  | <code>s1</code> 是 <code>s2</code> 的子集                                          |
| <code>s1 &lt; s2</code>   | <code>s1</code> 是 <code>s2</code> 的真子集                                         |
| <code>s1 &gt;= s2</code>  | <code>s1</code> 是 <code>s2</code> 的超集                                          |
| <code>s1 &gt; s2</code>   | <code>s1</code> 是 <code>s2</code> 的真超集 ( <code>s1</code> 不等于 <code>s2</code> ) |
| <code>s1   s2</code>      | <code>s1</code> 与 <code>s2</code> 的并集                                          |
| <code>s1 &amp; s2</code>  | <code>s1</code> 与 <code>s2</code> 的交集                                          |
| <code>s1 - s2</code>      | <code>s1</code> 与 <code>s2</code> 的差集                                          |
| <code>s1 ^ s2</code>      | 对称差分 (该集合中的元素在 <code>s1</code> 与 <code>s2</code> 的其中之一)                        |

# 字典运算符

- dict支持的运算符如下:

|                              |                                           |
|------------------------------|-------------------------------------------|
| <code>d[key]</code>          | 给定键 <code>key</code> 所关联的值                |
| <code>d[key] == value</code> | 设置（或重置）与给定的键相关联的值                         |
| <code>del d[key]</code>      | 从字典中删除键及其关联的值                             |
| <code>key in d</code>        | 检查 <code>key</code> 是 <code>d</code> 的成员  |
| <code>key not in d</code>    | 检查 <code>key</code> 不是 <code>d</code> 的成员 |
| <code>d1 == d2</code>        | <code>d1</code> 等价于 <code>d2</code>       |
| <code>d1 != d2</code>        | <code>d1</code> 不等价于 <code>d2</code>      |

# 扩展赋值运算符

- Python支持对大多数二元运算符的扩展赋值运算
  - 如`count+=5`
- 扩展运算符与略繁琐的表达式有时有语义方面的微妙差异：

```
alpha = [1, 2, 3]
beta = alpha # an alias for alpha
beta += [4, 5] # extends the original list with two more elements
beta = beta + [6, 7] # reassigns beta to a new list [1, 2, 3, 4, 5, 6, 7]
print(alpha) # will be [1, 2, 3, 4, 5]
```

# 运算符优先级

| 运算符优先级 |                        |                                              |
|--------|------------------------|----------------------------------------------|
|        | 类 型                    | 符 号                                          |
| 1      | 成员访问                   | expr.member                                  |
| 2      | 函数 / 方法调用<br>容器下标 / 切片 | expr(...)<br>expr[...]                       |
| 3      | 幂运算                    | **                                           |
| 4      | 一元运算符                  | + expr, - expr, ~ expr                       |
| 5      | 乘法, 除法                 | *, /, //, %                                  |
| 6      | 加法, 减法                 | +, -                                         |
| 7      | 按位移位                   | <<, >>                                       |
| 8      | 按位与                    | &                                            |
| 9      | 按位异或                   | ^                                            |
| 10     | 按位或                    |                                              |
| 11     | 比较<br>包含               | is, is not, ==, !=, <, <=, >, >=, in, not in |
| 12     | 逻辑非                    | not expr                                     |
| 13     | 逻辑与                    | and                                          |
| 14     | 逻辑或                    | or                                           |
| 15     | 条件判断                   | val1 if cond else val2                       |
| 16     | 赋值                     | =, +=, -=, *= 等                              |

# 运算符优先级

- 记不住优先级了怎么办?

## 打括号





# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- **控制流程**
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 控制流程

- 冒号字符用于表示代码块的开始，代码块作为控制结构的结构体
- 如果结构体可以被表述为一个可执行语句，则其可以与冒号置于同一行上，且在冒号右边
- 更通常的情况是，结构体从冒号的下一行开始整体缩进
- Python依赖于缩进级别或嵌套结构来指定代码块

# 条件语句

- Python中，条件语句的一般形式如下：

```
if first_condition:
 first_body
elif second_condition:
 second_body
elif third_condition:
 third_body
else:
 fourth_body
```

# 循环语句

- while循环:

```
while condition:
 body
```

- for循环:

```
for element in iterable:
 body
```

# body may refer to 'element' as an identifier

- 基于索引的for循环:

```
big_index = 0
for j in range(len(data)):
 if data[j] > data[big_index]:
 big_index = j
```

# break和continue语句

- Python支持break语句，当在循环体内执行break语句时，循环立即终止，如：

```
found = False
for item in data:
 if item == target:
 found = True
 break
```

- Python也支持continue语句，continue语句会使循环体的当前迭代终止，但其后续迭代正常进行

# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- **函数**
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 函数

- Python中的函数使用关键字def来定义：

```
def count(data, target):
 n = 0
 for item in data:
 if item == target:
 n += 1
 return n
```

- 以上的例子建立一个函数，其标识符为函数名count，并设立了期望的参数个数
- 第一行为函数的签名，函数定义的其余部分为函数的主体
- return语句用来表示函数应立即停止执行，并将所得到的值返回给调用者

# 函数的信息传递

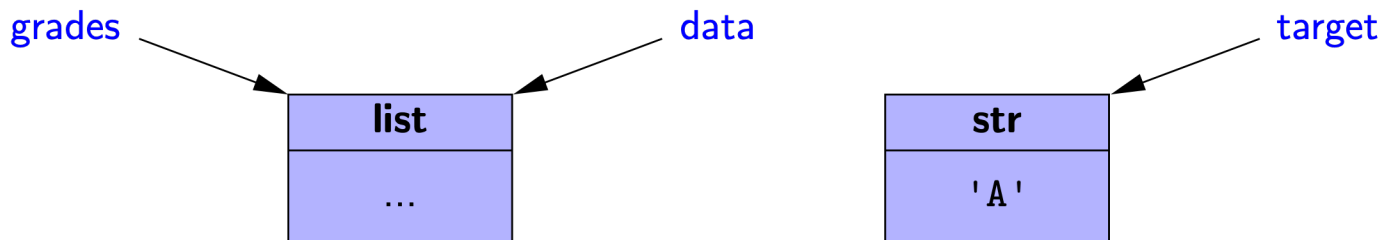
- Python中参数的传递遵循标准赋值语句的语法
- 例：以下语句执行时：

```
prizes = count(grades, 'A')
```

实际参数grades和'A'被隐式分配给形式参数：

```
data = grades
target = 'A'
```

其产生的效果为：





# 函数的信息传递

- 函数可以为参数声明一个或多个默认值，如：

```
def foo(a, b=15, c=27):
```

- 默认参数只能排在非默认参数后面
- Python支持使用关键字参数调用函数

- 例：如果函数签名为：

```
foo(a = 10, b = 20, c = 30)
```

调用foo(c = 5)即可将函数以参数a = 10, b = 20, c = 5的形式执行



# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 控制台输入输出

- 内置函数`print`用来生成标准输出到控制台
- 默认情况下，`print`函数输出时会在每对参数之间插入空格作为分隔符
- 例如，命令`print('maroon', 5)`会输出字符串`'maroon 5\n'`.
- 非字符串参数会以`str(x)`的形式显示

# 控制台输入输出

- Python使用内置函数来接收来自用户控制台的信息
- 如给出一个可选参数, 那么此函数会显示提示信息, 然后等待用户输入任意字符, 直到按下回车键
- 函数的返回值为回车键前用户输入的所有字符串
  - 此字符串可以被立即操作:

```
year = int(input('In what year were you born? '))
```

# 控制台输入输出

- 以下为一个简单但完整的使用print和input函数的示例：

```
age = int(input('Enter your age in years: '))
max_heart_rate = 206.9 - (0.67 * age) # as per Med Sci Sports Exerc.
target = 0.65 * max_heart_rate
print('Your target fat-burning heart rate is', target)
```

# 文件

- Python中访问文件要调用一个内置函数`open`，它返回一个和底层文件交互的对象
  - 例：`fp = open('sample.txt')`
- 文件的常用方法：

| 调用方法                               | 描 述                                               |
|------------------------------------|---------------------------------------------------|
| <code>fp.read()</code>             | 将可读文件剩下的所有内容作为一个字符串返回                             |
| <code>fp.read(k)</code>            | 将可读文件中接下来的 <code>k</code> 个字节作为一个字符串返回            |
| <code>fp.readline()</code>         | 从文件中读取一行内容，并以此作为一个字符串返回                           |
| <code>fp.readlines()</code>        | 将文件中的每行内容作为一个字符串存入列表中，并返回该列表                      |
| <code>for line in fp</code>        | 遍历文件的每一行                                          |
| <code>fp.seek(k)</code>            | 将当前位置定位到文件的第 <code>k</code> 个字节                   |
| <code>fp.tell()</code>             | 返回当前位置偏离开始处的字节数                                   |
| <code>fp.write(string)</code>      | 在可写文件的当前位置将 <code>string</code> 的内容写入             |
| <code>fp.writelines(seq)</code>    | 在可写文件的当前位置写入给定序列的每个字符串。除了那些嵌入到字符串中的换行符，这个命令不插入换行符 |
| <code>print(..., file = fp)</code> | 将 <code>print</code> 函数的输出重定向给文件（输出文件内容）          |

# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点



# 异常处理

- 异常是程序执行期间发生的突发性事件
  - 逻辑错误或未预料到的情况都有可能造成异常
- Python中，异常（也被称为**错误**）也是执行代码时遇到突发状况所引发（或抛出）的对象
  - Python解释器也可以引发异常
- 如在上下文中有处理异常的代码，则异常可能被捕获。否则，异常会导致解释器停止运行程序并向控制台发送合适的信息

# 常见异常类

- Python有大量的异常类，它们定义了各种不同类型的异常。下表中是常见异常类：

表 1-6 Python 中的常见异常类

| 异常类名              | 描 述                                |
|-------------------|------------------------------------|
| Exception         | 所有异常类的基类                           |
| AttributeError    | 如果对象 obj 没有 foo 成员，会由语法 obj.foo 引发 |
| EOFError          | 一个“end of file”到达控制台或者文件输入引发错误     |
| IOError           | 输入 / 输出操作（如打开文件）失败引发错误             |
| IndexError        | 索引超出序列范围引发错误                       |
| KeyError          | 请求一个不存在的集合或字典关键字引发错误               |
| KeyboardInterrupt | 用户按 ctrl - C 中断程序引发错误              |
| NameError         | 使用不存在的标识符引发错误                      |
| StopIteration     | 下一次遍历的元素不存在时引发错误，参照 1.8 节          |
| TypeError         | 发送给函数的参数类型不正确引发错误                  |
| ValueError        | 函数参数值非法时引发错误（例如，sqrt(-5)）          |
| ZeroDivisionError | 除数为 0 引发错误                         |

# 抛出异常

- 执行raise语句会抛出异常，并将异常类的相应实例作为指定问题的参数
  - 例：如果一个计算平方根的函数传递了一个负数作为参数，则会引发如下异常：

```
raise ValueError('x cannot be negative')
```

# 捕获异常

- Python中异常可以被try-except控制结构来测试及捕获:

```
try:
 ratio = x / y
except ZeroDivisionError:
 ... do something else ...
```

- 在这种结构中，try块中的代码是要被执行的初始代码
- Try块后通常跟着一个或多个except语句。这些语句会在try块引发指定异常后根据异常类型来执行



# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 迭代器

- Python的基本容器类型，如列表、元组和集合，都是可迭代的类型，可用在如下语句中

*for element in iterable:*

- 一个迭代器是一个对象，其通过一系列的值来管理迭代。如果定义变量*i*为一个迭代器对象，则*next(i)*会产生一个当前序列的下一个元素；如果没有后续元素，则抛出*StopIteration*异常
- 如一个对象*obj*是可迭代的，则*iter(obj)*产生一个迭代器.

# 生成器

- Python中最方便的创建迭代器的方式时使用生成器
- 生成器的语法类似于函数，但不使用return返回值，而是通过yield来展示序列中的每一个元素
- 例：一个计算n的所有因子的生成器：

```
def factors(n): # generator that computes factors
 for k in range(1,n+1):
 if n % k == 0: # divides evenly, thus k is a factor
 yield k # yield this factor as next result
```

- 应用场景：如"for factor in factors(100):"



# Python程序设计回顾

- Python概述
- Python对象及数据类型
- 表达式、运算符和优先级
- 控制流程
- 函数
- 简单的输入输出
- 异常处理
- 迭代器和生成器
- Python的其他特点

# 条件表达式

- Python支持使用条件表达式来代替一个简单的控制结构，其一般的语法形式如下：

*expr1 if condition else expr2*

- 这种复合表达式，如condition为真，则计算expr1，否则计算expr2
- 例：

```
param = n if n >= 0 else -n # pick the appropriate value
result = foo(param) # call the function
```

- 或：

```
result = foo(n if n >= 0 else -n)
```

# 解析语法

- 一个很常见的编程任务是基于一个序列的处理来产生另一个序列的值
- 通常，这个任务在Python中使用所谓的解析语法可以简单地实现：

`[ expression for value in iterable if condition ]`

- 这一语句与以下的控制结构计算的列表是相同的：

```
result = []
for value in iterable:
 if condition:
 result.append(expression)
```

# 序列类型的打包

- 如果在大的上下文中给出了一系列逗号分隔的表达式，它们将被视为一个单独的元组，即使没有圆括号

- 例：

```
data = 2, 4, 6, 8
```

- 这将标识符data赋值为元组 (2, 4, 6, 8)，这叫做元组的自动打包

# 序列类型的解包

- 作为打包的对偶行为，Python也可以自动解包一个序列，允许单个标识符的一系列元素赋值给序列中的各个元素
- 例：

```
a, b, c, d = range(7, 11)
```

- 这一语句同时实现了a=7，b=8，c=9和d=10

# 模块和import语句

- 除了内置的定义外，标准的Python包含数以千计的被组织在附加库中的数值、函数类。这些库被称为模块，可以通过如下的import语句导入：

```
import math
```

# 常用现有模块

表 1-7 一些与数据结构和算法相关的现有 Python 模块

| 模块名         | 描述                       |
|-------------|--------------------------|
| array       | 为原始类型提供了紧凑的数组存储          |
| collections | 定义额外的数据结构和包括对象集合的抽象基类    |
| copy        | 定义通用函数来复制对象              |
| heapq       | 提供基于堆的优先队列函数（参见 9.3.7 节） |
| math        | 定义常见的数学常数和函数             |
| os          | 提供与操作系统交互                |
| random      | 提供随机数生成                  |
| re          | 对处理正则表达式提供支持             |
| sys         | 提供了与 Python 解释器交互的额外等级   |
| time        | 对测量时间或延迟程序提供支持           |





# 面向对象编程



# 面向对象编程

- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝

# 面向对象编程

- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝

# 类、对象与实例

- 每个对象都是类的一个实例
- 每个类呈现给外部世界的是该类实例的一种简洁、一致的概括，没有太多对象内部的不必要的细节
- 类的定义通常详细规定了对象所包含的实例变量（或称为数据成员）；还规定类对象中可执行的方法（或称为成员函数）

# 面向对象的设计目标

- 健壮性
  - 我们希望软件能处理我们在程序中没有明确定义的异常输入
- 适应性
  - 软件需要随着时间不断地优化，以应对外部环境中条件的改变
- 可重用性
  - 同样的代码可以用在不同系统的各种应用中

# 面向对象的设计原则

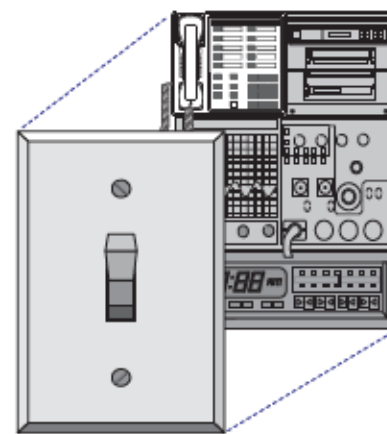
- 模块化 (modularity)
  - 在这一原则下，不同组件归位不同的功能单元
- 抽象化 (Abstraction)
  - 抽象数据类型 (ADT)、抽象基类 (ABC)
- 封装 (Encapsulation)



Modularity



Abstraction



Encapsulation

# 面向对象的设计





# 面向对象的设计





# 面向对象的设计



# 抽象数据类型 (ADT)

- 抽象化是指从一个系统中提炼出其最基本的部分
- 将抽象模式应用于数据结构的设计便产生了抽象数据类型 (Abstract Data Types, ADT)
- ADT是数据结构的一个模型。它规定了数据存储的类型、支持的操作、以及操作的参数的类型.
- ADT定义了每个操作要做什么，而不是怎么做
- ADT所支持的行为的集合即其公共接口

# ADT例子：数组

ADT Array {

数据对象：  $A = \{A_1, A_2, \dots, A_N\}$

基本操作：

A.init(S): 使用序列S初始化数组A

len(A): 返回数组A的长度

A.is\_empty(): 检查数组A是否为空，如为空则返回True

A.get\_item(n): 返回数组A的第n个元素

A.locate(e): 返回数组A中第一个等于e的元素的位置

A.insert(n,e): 在数组A的第n个元素前插入元素e

A.delete(n): 删除数组A的第n个元素

A.clear(): 清空数组A

} ADT Array

# 封装

- 封装是面向对象设计的另一个重要原则
  - 软件系统的不同组件不应显示其各自实现的内部细节
- 一个数据结构的一些方面会被认定为公共部分，而另一些会被认定为内部细节
- Python仅对封装提供较为宽松的支持
  - 按照惯例，以单下划线开头的类成员（数据成员和成员函数）被认定为非公开的

# 面向对象的设计

- 责任：把工作分为不同的角色，它们有不同的责任
- 独立：尽可能将各类以独立于其他类的原则进行定义
- 行为：仔细且精准地定义每个类的行为，这样与它交互的其他类可以很好地理解由这个类执行所得到的结果

# UML图

- UML图是一种表达面向对象设计的标准视觉记号
- 类图是一种UML图，它有三部分：
  - 类名
  - 建议的实例变量（域）
  - 建议的类的方法

|     |                                                                       |                                               |
|-----|-----------------------------------------------------------------------|-----------------------------------------------|
| 类:  | 信用卡                                                                   |                                               |
| 域:  | _customer<br>_bank<br>_account                                        | _balance<br>_limit                            |
| 行为: | get_customer()<br>get_bank()<br>get_account()<br>make_payment(amount) | get_balance()<br>get_limit()<br>charge(price) |



# 面向对象编程

- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝



# 类定义

- 类是面向对象程序设计中抽象的主要方法
- 在Python中，每一块数据被以类的实例（即对象）的方式所表示
- 类及它的所有实例给成员函数（也称为方法）提供了一系列行为
- 类有效率地以属性（也称为域，实例变量，或数据成员）的形式决定每个实例状态信息的表示方式

# self标识符

- 在Python的类中，**self**标识符扮演着重要的角色
- 每个类可以有許多不同的实例，而每个实例都需要存储自己的实例变量
- 因此，每个实例存储能反映其目前状态的实例变量。从语句构成上来讲，**self**标识了正在调用某一方法的实例

# 构造函数

- 用户可以通过类似下面的语法创建CreditCard类的实例：

```
cc = CreditCard('John Doe', '1st Bank', '5391 0375 9387 5309', 1000)
```

- 这一语句调用一个名为\_\_init\_\_的特殊方法来构造一个新的实例，这一方法被称为构造函数
- 构造函数的主要任务是用合适的实例变量来建立一个新创建的对象

## 例：CreditCard类

```
1 class CreditCard:
2 """A consumer credit card."""
3
4 def __init__(self, customer, bank, acct, limit):
5 """Create a new credit card instance.
6
7 The initial balance is zero.
8
9 customer the name of the customer (e.g., 'John Bowman')
10 bank the name of the bank (e.g., 'California Savings')
11 acct the account identifier (e.g., '5391 0375 9387 5309')
12 limit credit limit (measured in dollars)
13 """
14 self._customer = customer
15 self._bank = bank
16 self._account = acct
17 self._limit = limit
18 self._balance = 0
19
```

## 例：CreditCard类

```
20 def get_customer(self):
21 """Return name of the customer."""
22 return self._customer
23
24 def get_bank(self):
25 """Return the bank's name."""
26 return self._bank
27
28 def get_account(self):
29 """Return the card identifying number (typically stored as a string)."""
30 return self._account
31
32 def get_limit(self):
33 """Return current credit limit."""
34 return self._limit
35
36 def get_balance(self):
37 """Return current balance."""
38 return self._balance
```

## 例：CreditCard类

```
39 def charge(self, price):
40 """ Charge given price to the card, assuming sufficient credit limit.
41
42 Return True if charge was processed; False if charge was denied.
43 """
44 if price + self._balance > self._limit: # if charge would exceed limit,
45 return False # cannot accept charge
46 else:
47 self._balance += price
48 return True
49
50 def make_payment(self, amount):
51 """ Process customer payment that reduces balance. """
52 self._balance -= amount
```

# 运算符重载

- Python的内置类为许多操作符定义类自然的语义
  - 例： $a + b$ 对数值类型定义为加法，而对序列类型定义为拼接
- 定义一个新类时，我们必须考虑当 $a$ 或 $b$ 是类的一个实例时， $a + b$ 这样的语句是否应该被定义
- 在类中，可以通过运算符重载来定义已有的运算符
  - 如“+”运算符可以通过`__add__`方法进行重载，重载后 $a + b$ 的操作将通过调用`a.__add__(b)`来进行
  - 其他常用运算符也可通过这种方式进行重载

# 类中的迭代器

- 迭代器是数据结构的设计中一个重要的概念
- 一个可迭代对象的迭代器提供了一个关键的行为：
  - 它支持一个特殊的方法\_\_next\_\_。如果可迭代对象有下一个元素，则此方法返回该元素；否则，它抛出一个StopIteration异常来表明没有下一个元素



# 自动迭代器

- Python也为实现了\_\_len\_\_和\_\_getitem\_\_方法的类提供一个自动的迭代器
- 例：Range类

```
1 class Range:
2 """A class that mimic's the built-in range class."""
3
4 def __init__(self, start, stop=None, step=1):
5 """Initialize a Range instance.
6
7 Semantics is similar to built-in range class.
8 """
9 if step == 0:
10 raise ValueError('step cannot be 0')
11
12 if stop is None: # special case of range(n)
13 start, stop = 0, start # should be treated as if range(0,n)
14
15 # calculate the effective length once
16 self._length = max(0, (stop - start + step - 1) // step)
17
18 # need knowledge of start and step (but not stop) to support __getitem__
19 self._start = start
20 self._step = step
21
22 def __len__(self):
23 """Return number of entries in the range."""
24 return self._length
25
26 def __getitem__(self, k):
27 """Return entry at index k (using standard interpretation if negative)."""
28 if k < 0:
29 k += len(self) # attempt to convert negative index
30
31 if not 0 <= k < self._length:
32 raise IndexError('index out of range')
33
34 return self._start + k * self._step
```



# 面向对象编程

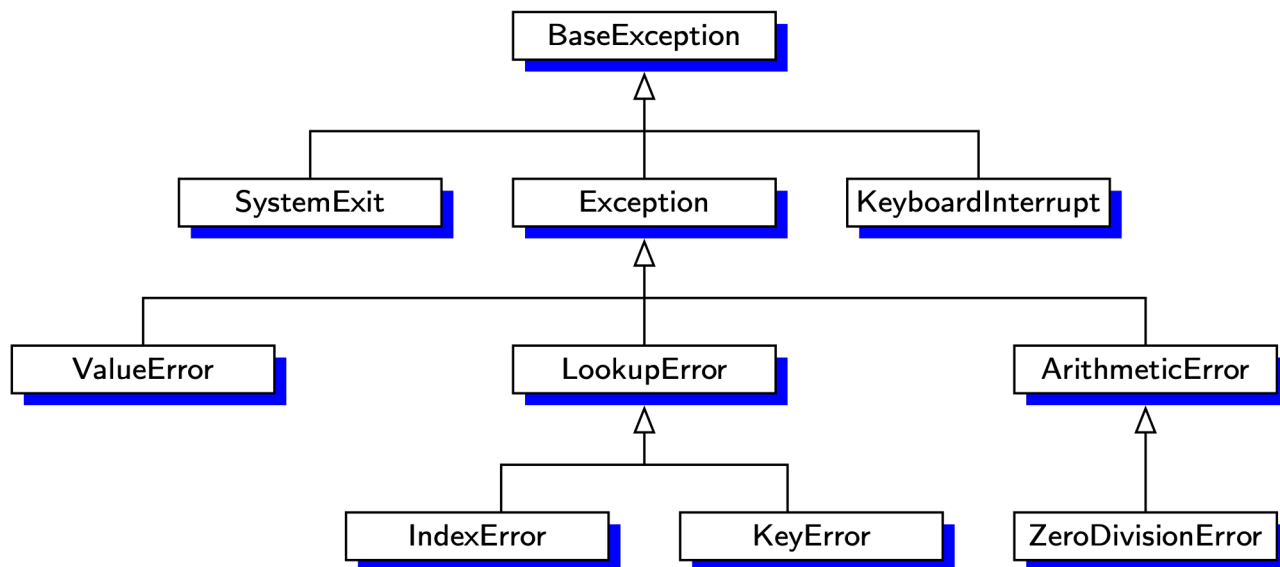
- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝

# 继承

- 模块化和层次化组织中的一个机制是**继承**
- 继承允许基于一个现有的类来定义新的类
- 通常，现有的类被称为基类（base class），父类（parent class）或超类（superclass）；新的类被称为子类（subclass 或child class）
- 有两种方式可以让子类有别于父类：
  - 提供一个新的覆盖现有方法的对父类行为的特化方法
  - 提供新的方法

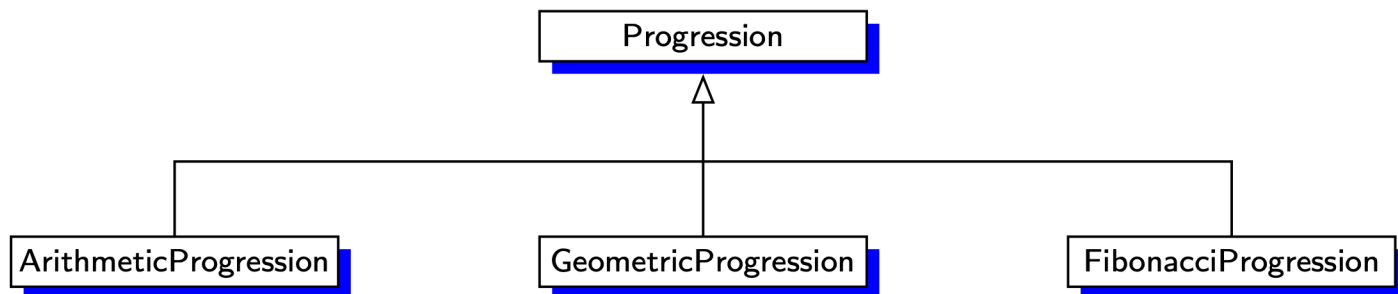
# Python的异常层次结构

- Python异常类型层次的一部分：



# 例：数列的层次图

- 一个数列是一个数的序列，其中每个数都依赖于之前的一个或多个数
  - 等差数列通过给前一个数加上一个固定常量得到下一个数
  - 等比数列通过给前一个数乘一个固定常量得到下一个数
  - 斐波那契数列中的数通过以下递推公式得出： $N_{i+1}=N_i+N_{i-1}$



# 数列的基类

```
1 class Progression:
2 """Iterator producing a generic progression.
3
4 Default iterator produces the whole numbers 0, 1, 2, ...
5 """
6
7 def __init__(self, start=0):
8 """Initialize current to the first value of the progression."""
9 self._current = start
10
11 def _advance(self):
12 """Update self._current to a new value.
13
14 This should be overridden by a subclass to customize progression.
15
16 By convention, if current is set to None, this designates the
17 end of a finite progression.
18 """
19 self._current += 1
```

# 数列的基类

```
20
21 def __next__(self):
22 """Return the next element, or else raise StopIteration error."""
23 if self._current is None: # our convention to end a progression
24 raise StopIteration()
25 else:
26 answer = self._current # record current value to return
27 self._advance() # advance to prepare for next time
28 return answer # return the answer
29
30 def __iter__(self):
31 """By convention, an iterator must return itself as an iterator."""
32 return self
33
34 def print_progression(self, n):
35 """Print next n values of the progression."""
36 print(' '.join(str(next(self)) for j in range(n)))
```



# 等差数列类

```
1 class ArithmeticProgression(Progression): # inherit from Progression
2 """Iterator producing an arithmetic progression."""
3
4 def __init__(self, increment=1, start=0):
5 """Create a new arithmetic progression.
6
7 increment the fixed constant to add to each term (default 1)
8 start the first term of the progression (default 0)
9 """
10 super().__init__(start) # initialize base class
11 self._increment = increment
12
13 def _advance(self): # override inherited version
14 """Update current value by adding the fixed increment."""
15 self._current += self._increment
```

# 等比数列类

```
1 class GeometricProgression(Progression): # inherit from Progression
2 """Iterator producing a geometric progression."""
3
4 def __init__(self, base=2, start=1):
5 """Create a new geometric progression.
6
7 base the fixed constant multiplied to each term (default 2)
8 start the first term of the progression (default 1)
9 """
10 super().__init__(start)
11 self._base = base
12
13 def _advance(self): # override inherited version
14 """Update current value by multiplying it by the base value."""
15 self._current *= self._base
```

# 斐波那契数列类

```
1 class FibonacciProgression(Progression):
2 """Iterator producing a generalized Fibonacci progression."""
3
4 def __init__(self, first=0, second=1):
5 """Create a new fibonacci progression.
6
7 first the first term of the progression (default 0)
8 second the second term of the progression (default 1)
9 """
10 super().__init__(first) # start progression at first
11 self._prev = second - first # fictitious value preceding the first
12
13 def _advance(self):
14 """Update current value by taking sum of previous two."""
15 self._prev, self._current = self._current, self._prev + self._current
```

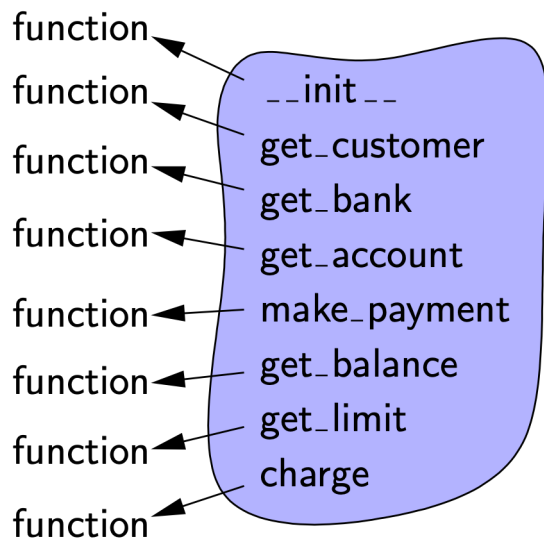


# 面向对象编程

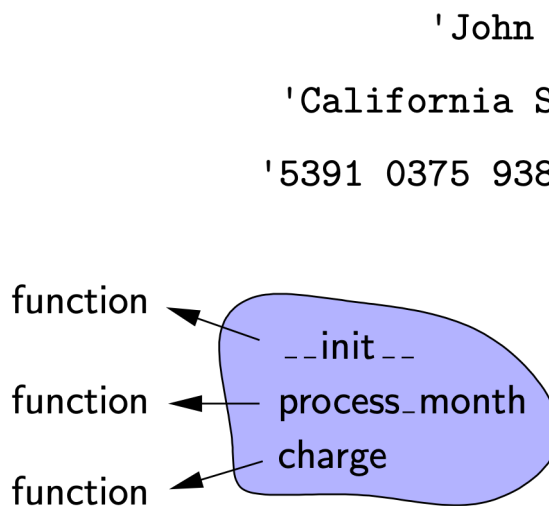
- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝

# 实例和类命名空间

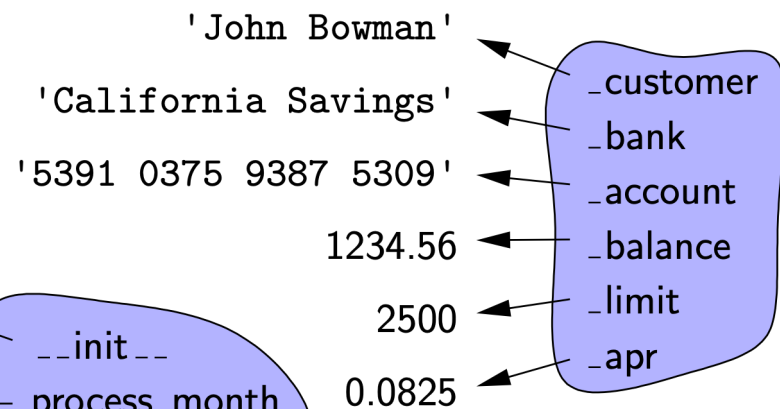
- 每个对象有其实例命名空间，类也有单独的类命名空间
  - 对象的命名空间包含对象的数据成员
  - 类的命名空间包含类的方法，所有类的实例共用这些方法



Credit Card类  
命名空间



Predatory  
Credit Card类  
命名空间



Predatory Credit  
Card对象的实例  
命名空间

# 实例和类命名空间

- 当一些值被一个类的所有实例共享时，可以定义类级别的数据成员

```
class PredatoryCreditCard(CreditCard):
 OVERLIMIT_FEE = 5 # this is a class-level member

 def charge(self, price):
 success = super().charge(price)
 if not success:
 self._balance += PredatoryCreditCard.OVERLIMIT_FEE
 return success
```

# 嵌套类

- 在一个类的范围内嵌套地定义的另一类，被称为嵌套类
- 嵌套类有助于减少潜在的命名冲突
- 外部类的子类可以重载嵌套类的定义

```
class A: # the outer class
 class B: # the nested class
 ...
```



## \_\_slots\_\_声明

- 通常，Python中的每一个命名空间均代表内置dict类的一个实例
- Python提供一种更直接的机制来表示实例的命名空间以在有大量此类的实例时节省存储空间，即使用\_\_slots\_\_声明

```
class CreditCard:
```

```
 __slots__ = '_customer', '_bank', '_account', '_balance', '_limit'
```

# 面向对象编程

- 面向对象的设计
- 类定义
- 类的继承
- 命名空间
- 深拷贝与浅拷贝

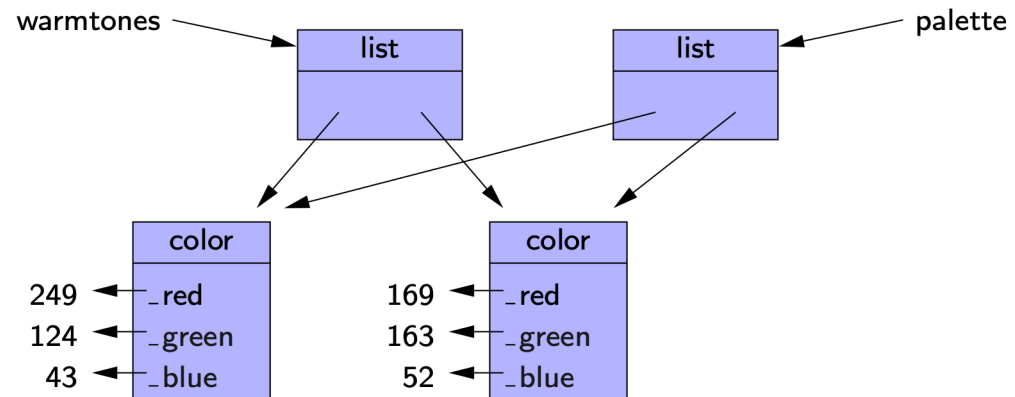
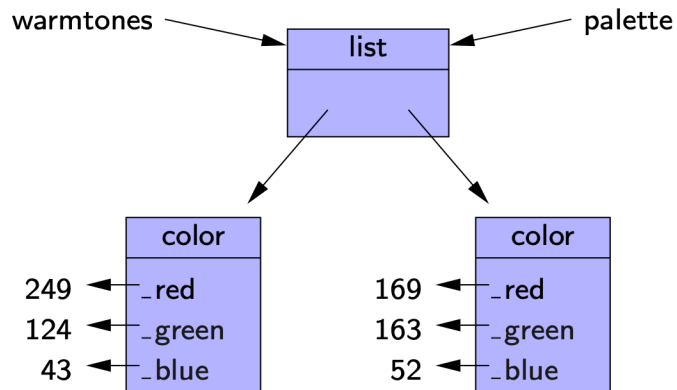
# 深拷贝与浅拷贝

- 直接赋值的结果是产生变量的一个别名，如

`palette = warmtones`

- 如果调用列表的构造函数产生新的列表，则产生一个新列表，这被称为浅拷贝

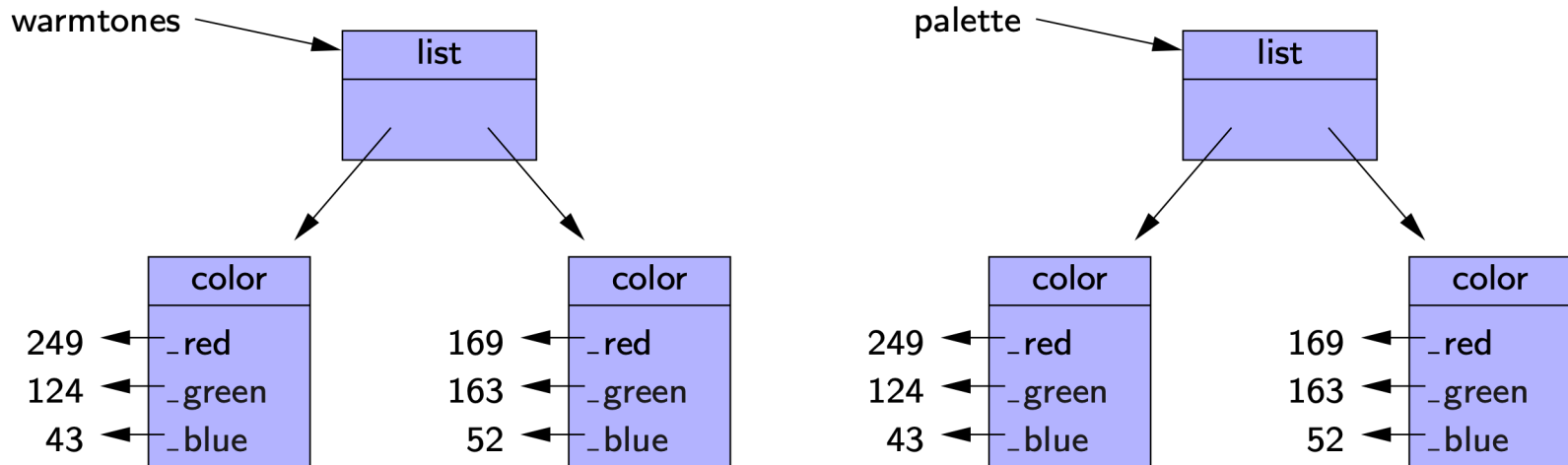
`palette = list(warmtones)`



# 深拷贝与浅拷贝

- 我们希望的操作是深拷贝，即新实例和旧实例没有关系，这可以通过copy模块的deepcopy函数完成

```
palette = copy.deepcopy(warmtones)
```





*Enjoy*